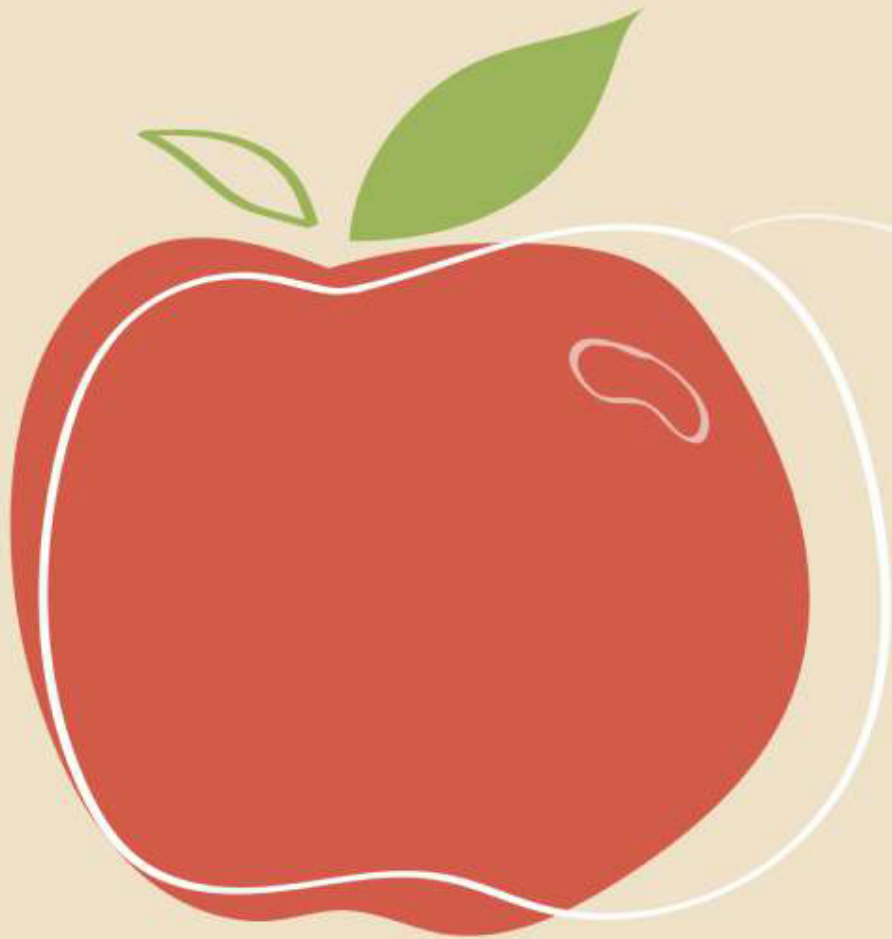


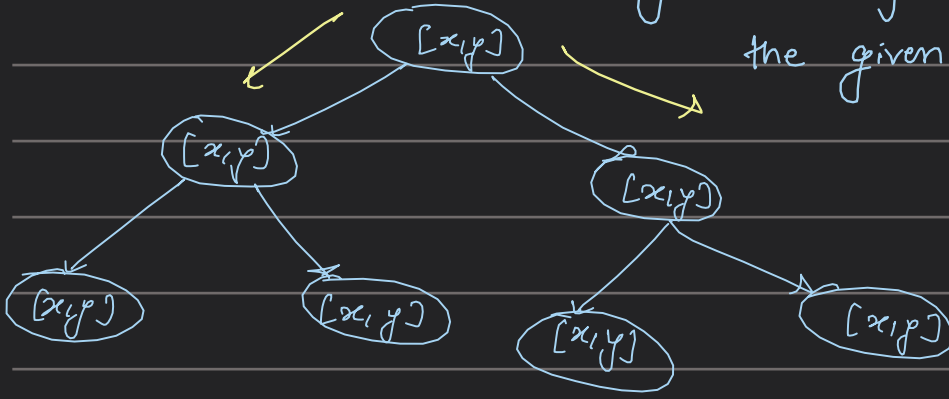
SLAM-Checkpoint-3

-KD-Trees



KD - Trees

using BST to find the closest point
the given point

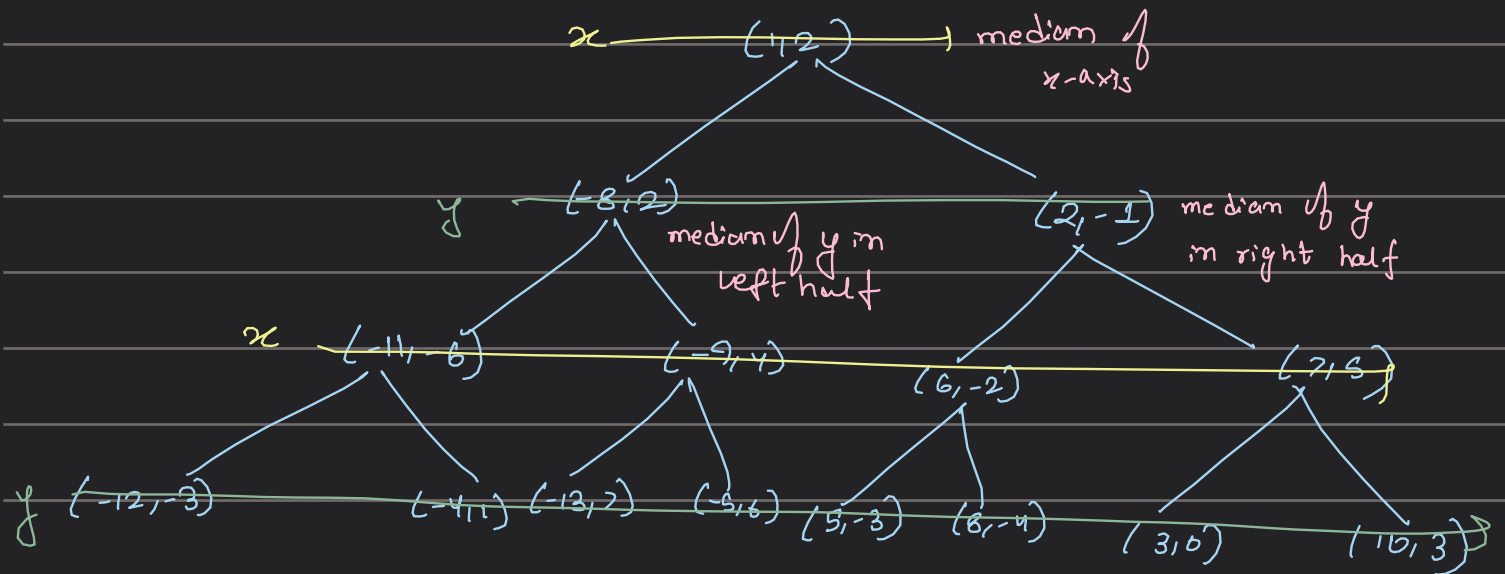


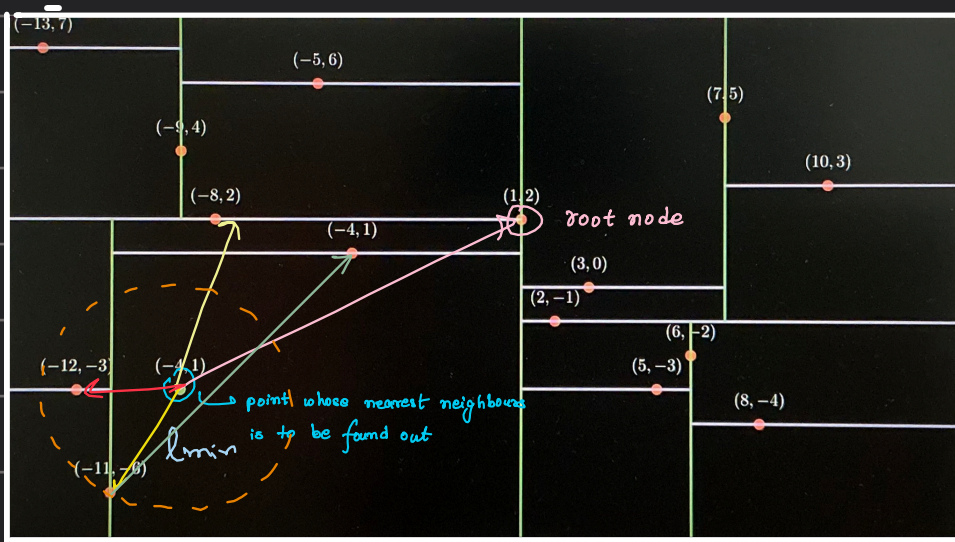
Building a KD-tree $\hookrightarrow$ find median, split data, repeat

① first split from the x-axis

$\hookrightarrow$ splitting about the
median of x-axis

② then split from the y-axis $\hookrightarrow$ splitting about the median of
y-axis
 $\downarrow$
repeat





① calculate the dist. b/w root node & the given point

② the point lies on the left side so cal. the dist. with the left child.

③ Repeat this until we reach the leaf node of that side

④ the leaf node has larger dist. than the parent node so the previous parent node becomes the nearest ($dist = lmin$)

⑤ Draw a circle with $r = lmin$ & move in $+x/-x$ & $+y/-y$ & compare the dist. from the first point in that direction from $lmin$

⑥ So in this case $(-12, -3)$ comes out to be the closest.

```
import os
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from scipy.spatial import distance
import pandas as pd
from scipy.spatial import cKDTree
```

```
225  def KDTree(perceived_cones, real_cones, threshold=None):
226      tree = cKDTree(real_cones)
227      dists, indices = tree.query(perceived_cones, k=1)
228
229      if threshold is not None:
230          indices = np.where(dists <= threshold, indices, -1)
231
232      return indices
233
234  def main():
235      print("\nComputing KD-Tree NN matches...")
236
237      kdtree_matches = KDTree(
238          BLUE_CONES,
239          YELLOW_CONES,
240          threshold=5
241      )
242
243      print("KD-Tree Indices:")
244      print(kdtree_matches)
245
```